Java avancé et interfaces graphiques

Informatique — Licence 2 (L2)

Cours INF-L2-14

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Donner une interface graphique à tes applications Java : organisation du code en packages, fenêtres JavaFX (conteneurs, contrôles, événements, TableView), connexion à une base de données MySQL avec JDBC et requêtes préparées, puis empaquetage et distribution d'une application de bureau complète et documentée.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Rappels de POO et organisation du code en packages
• Leçon 2 — Interfaces graphiques avec JavaFX : conteneurs et contrôles
• Leçon 3 — Gestion des événements dans une interface graphique
• Leçon 4 — Tableaux, listes et formulaires graphiques
• Leçon 5 — Connecter une application Java à une base de données avec JDBC
• Leçon 6 — Empaquetage et distribution d'une application Java
• Leçon 7 — Projet : application de bureau complète et documentée
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Dans le cours INF-L2-01, tu as appris la programmation orientée objet en Java avec des programmes qui dialoguaient dans la console : on tapait des valeurs au clavier et les réponses s'affichaient en texte. C'est parfait pour apprendre, mais aucun utilisateur final n'acceptera de gérer une bibliothèque, une scolarité ou un stock de cette façon. Les applications de bureau professionnelles présentent des fenêtres, des boutons, des tableaux et des formulaires : c'est exactement ce que ce cours va te faire construire.

Notre fil rouge sera BiblioIAM, une application de gestion du catalogue de la bibliothèque de l'IAM. Nous la construirons couche par couche : d'abord un modèle métier organisé en packages, puis une fenêtre JavaFX, des réactions aux clics, un tableau de livres relié aux objets, une vraie base de données MySQL interrogée en JDBC, et enfin l'empaquetage de l'application pour la donner à un utilisateur. Chaque leçon ajoute une couche au même projet, comme dans un vrai développement.

Ce cours suppose que tu maîtrises les bases de la POO (classes, encapsulation, héritage, exceptions, collections) vues en INF-L2-01, ainsi que les requêtes SQL du cours INF-L2-04. Les exemples de code sont courts et complets : retape-les, lance-les, puis modifie-les. C'est en cassant et en réparant une fenêtre que les mécanismes de JavaFX et de JDBC deviennent évidents.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

• Organiser une application Java en packages cohérents (modèle, accès aux données, interface) et appliquer les règles de visibilité entre packages.

• Construire une fenêtre JavaFX structurée avec Stage, Scene et les conteneurs BorderPane, VBox, HBox et GridPane.

• Réagir aux actions de l'utilisateur grâce aux événements et aux gestionnaires écrits en lambda (setOnAction).

• Afficher et modifier des objets dans un TableView, construire des formulaires de saisie et gérer la sélection.

• Connecter l'application à une base MySQL via JDBC : connexion, PreparedStatement, ResultSet et try-with-resources.

• Empaqueter l'application (jar exécutable, modules, jlink, jpackage) et la distribuer à des utilisateurs sans Java installé.

• Réaliser et documenter une application de bureau complète, de la base de données à l'interface graphique.

Leçon 1 — Rappels de POO et organisation du code en packages

Un package est à la fois un dossier sur le disque et un espace de noms pour les classes. La première ligne d'un fichier Java déclare son package, par exemple « package com.iam.biblio.modele ; », et le fichier doit se trouver dans un dossier qui reflète ce chemin : com/iam/biblio/modele/. Par convention, on utilise le nom de domaine inversé de l'organisation pour éviter que deux bibliothèques écrites par des équipes différentes n'entrent en collision. Sans package, un projet qui grandit devient vite un sac de classes illisible où tout le monde touche à tout.

Le package contrôle aussi la visibilité. Une classe déclarée public est utilisable depuis n'importe quel autre package, à condition de l'importer (« import com.iam.biblio.modele.Livre ; »). Sans le mot-clé public, la classe reste visible uniquement dans son propre package. Les attributs, eux, restent private : c'est l'encapsulation déjà vue en L2 — on n'y accède que par les getters et les méthodes métier. Notre classe Livre illustre cette discipline : son identifiant est final (il ne change jamais), et l'état « disponible » ne peut changer que par les méthodes emprunter() et retourner(), qui contrôlent la cohérence.

Pour BiblioIAM, nous découpons l'application en trois packages. Le package modele contient les données métier (Livre, Etudiant...) sans aucune trace d'interface ni de SQL. Le package dao (Data Access Object) regroupe les classes qui parlent à la base de données. Le package ui contient les fenêtres JavaFX. La règle de dépendance est stricte : ui utilise dao, dao utilise modele, et jamais l'inverse. Ce découpage permet de tester le modèle sans base de données, de changer l'interface sans toucher au SQL, et de travailler en équipe sans se marcher dessus.

L'essentiel
Package = dossier + espace de noms + frontière de visibilité. Une application bien organisée sépare le modèle (données métier), le dao (accès base de données) et l'interface (ui), avec des dépendances dans un seul sens.

Exemple — La classe Livre, dans le package com.iam.biblio.modele

```
package com.iam.biblio.modele;

public class Livre {
private final int id;
private String titre;
private String auteur;
private boolean disponible;

public Livre(int id, String titre, String auteur) {
this.id = id;
this.titre = titre;
this.auteur = auteur;
this.disponible = true;
}

public int getId() { return id; }
public String getTitre() { return titre; }
public String getAuteur() { return auteur; }
public boolean isDisponible() { return disponible; }

public void emprunter() {
if (!disponible) {
throw new IllegalStateException("Livre déjà emprunté");
}
disponible = false;
}

public void retourner() { disponible = true; }

@Override
public String toString() {
return titre + " — " + auteur
+ (disponible ? " (disponible)" : " (emprunté)");
}
}
```

À retenir

- La déclaration package est la première ligne du fichier, et les dossiers doivent refléter son chemin.
- Une classe utilisée depuis un autre package doit être public et importée ; les attributs restent private.
- Architecture en couches : ui → dao → modele, jamais l'inverse.

Exercice

Sur le même modèle que Livre, écris dans le package com.iam.biblio.modele une classe Etudiant avec un matricule (final), un nom, un prénom et une classe (par exemple « L2 Informatique »), les getters, et une méthode toString() qui retourne par exemple « IAM-2026-014 — Awa Diallo (L2 Informatique) ». Écris ensuite, dans le

package com.iam.biblio, une classe MainEtudiant qui crée un étudiant et l'affiche.

✔ Corrigé
La classe Etudiant suit exactement la même structure que Livre : attributs private, matricule en final, un constructeur qui reçoit les quatre valeurs, quatre getters, et toString() qui concatène matricule, prénom, nom et classe. La classe MainEtudiant, placée dans un autre package (com.iam.biblio), doit importer la classe : « import com.iam.biblio.modele.Etudiant ; » — sans cet import, le compilateur répond « cannot find symbol ». Le programme affiche : IAM-2026-014 — Awa Diallo (L2 Informatique). C'est la vérification à faire à chaque fois : si ton code compile et affiche la ligne attendue, ton organisation en packages est correcte.

Leçon 2 — Interfaces graphiques avec JavaFX : conteneurs et contrôles

Une application JavaFX hérite de la classe Application et redéfinit la méthode start(Stage), que la bibliothèque appelle au lancement. Le Stage est la fenêtre elle-même (titre, taille, bouton fermer). On y place une Scene, qui contient tout le contenu visible. Le contenu est un arbre de nœuds (le graphe de scène) : au sommet un conteneur racine, et dedans les contrôles. Attention, depuis Java 11, JavaFX ne fait plus partie du JDK : c'est une bibliothèque séparée (le SDK JavaFX) qu'il faut ajouter au projet — nous verrons en leçon 6 comment cela influence le lancement et l'empaquetage.

On ne place presque jamais les contrôles avec des coordonnées absolues : on les range dans des conteneurs qui les disposent automatiquement et s'adaptent quand la fenêtre est redimensionnée. BorderPane découpe la fenêtre en cinq zones (haut, bas, gauche, droite, centre) — le squelette classique d'une application. VBox empile verticalement, HBox horizontalement, avec un espacement (spacing) et des marges internes (padding). GridPane place les éléments dans une grille de lignes et de colonnes, idéale pour les formulaires. Les contrôles de base de ce cours : Label (texte affiché), TextField (champ de saisie, avec un texte d'aide fantôme grâce à setPromptText) et Button.

L'exemple construit la première fenêtre de BiblioIAM : un titre en haut d'un BorderPane, un formulaire de saisie au centre (deux champs et un bouton dans un HBox), et un label de statut en bas. À ce stade, le bouton ne fait encore rien : une interface JavaFX se construit en deux temps, d'abord l'assemblage visuel, ensuite les réactions aux actions de l'utilisateur — ce sera la leçon 3. Retiens la mécanique finale de start() : on compose la racine, on crée la Scene avec ses dimensions, on la donne au Stage avec setScene, on fixe le titre avec setTitle, et on affiche avec show(). Sans show(), rien n'apparaît.

L'essentiel
Stage = la fenêtre, Scene = son contenu, organisé en arbre de nœuds. Les conteneurs (BorderPane, VBox, HBox, GridPane) disposent les contrôles automatiquement ; launch(args) dans main démarre l'application.

Exemple — Première fenêtre BiblioIAM : BorderPane, HBox, VBox et contrôles de base

```
package com.iam.biblio.ui;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class FenetreBiblio extends Application {
@Override
public void start(Stage fenetre) {
Label titre = new Label("BiblioIAM — Gestion des livres");
Label statut = new Label("Aucun livre ajouté pour le moment.");
TextField champTitre = new TextField();
champTitre.setPromptText("Titre du livre");
TextField champAuteur = new TextField();
champAuteur.setPromptText("Auteur");
Button boutonAjouter = new Button("Ajouter");
HBox ligneSaisie = new HBox(8, champTitre, champAuteur, boutonAjouter);
ligneSaisie.setPadding(new Insets(10));
VBox centre = new VBox(10, ligneSaisie, new Label("Formulaire d'ajout"));
centre.setPadding(new Insets(10));
BorderPane racine = new BorderPane();
racine.setTop(titre);
racine.setCenter(centre);
racine.setBottom(statut);
fenetre.setTitle("BiblioIAM");
fenetre.setScene(new Scene(racine, 520, 300));
fenetre.show();
}

public static void main(String[] args) {
launch(args);
}
}
```

À retenir

- start(Stage) est le point d'entrée : on y construit la scène puis on appelle show().
- Un contrôle ne s'affiche que s'il est ajouté dans un conteneur lui-même placé dans la scène.
- setPromptText affiche un texte d'aide dans un champ vide ; il ne remplace pas une étiquette.

Exercice

Modifie la fenêtre de l'exemple : ajoute un troisième champ de saisie « Année » dans le HBox (avec son prompt), change le titre de la fenêtre en « BiblioIAM — Catalogue », et remplace le texte du label central par « Nouveau livre ». Indique pour chaque modification la ligne exacte que tu touches.

✔ Corrigé
Trois modifications localisées. 1) Après la création de champAuteur, déclare : TextField champAnnee = new TextField(); champAnnee.setPromptText("Année"); puis ajoute champAnnee dans le constructeur du HBox, après champAuteur. 2) Le titre de la fenêtre se change à l'endroit où l'on appelle fenetre.setTitle(...). 3) Le label central est créé inline dans le VBox : c'est sa chaîne qu'il faut remplacer. Leçon de méthode : dans une interface construite en code, chaque élément visible correspond à une ligne identifiable — quand quelque chose ne s'affiche pas, cherche d'abord s'il a bien été créé, puis s'il a été ajouté à un conteneur placé dans la scène.

Leçon 3 — Gestion des événements dans une interface graphique

Un programme graphique ne suit pas un déroulement linéaire : il attend. Chaque action de l'utilisateur — clic, touche Entrée, frappe au clavier — produit un événement. Pour réagir, on enregistre un gestionnaire d'événement sur le contrôle concerné. Pour un bouton, la méthode pratique est setOnAction, qui reçoit un objet EventHandler<ActionEvent>. Comme EventHandler est une interface fonctionnelle (une seule méthode handle), on l'écrit presque toujours avec une expression lambda : bouton.setOnAction(e -> { ... }). Le paramètre e représente l'événement lui-même ; le plus souvent, on ne l'utilise même pas.

Dans le corps du gestionnaire, on lit l'état des contrôles (champ.getText()), on met à jour le modèle ou d'autres contrôles (label.setText(...)), puis l'interface se rafraîchit toute seule. C'est aussi là qu'on valide les saisies : trim() pour supprimer les espaces de bord, isEmpty() pour refuser un champ vide. Notre compteur d'emprunts illustre trois gestionnaires : le bouton « Emprunter » incrémente, le bouton « Réinitialiser » remet à zéro, et le champ de nom réagit à la touche Entrée — car un TextField déclenche lui aussi un ActionEvent quand on appuie sur Entrée, avec le même mécanisme setOnAction.

Où vit l'état de l'application entre deux événements ? Dans les attributs de la classe qui étend Application. Le compteur emprunts est un champ de la classe : chaque clic relit et modifie la même variable. Déclarer ce compteur comme variable locale de start() serait une erreur classique : la lambda ne pourrait même pas le modifier (une variable capturée par une lambda doit être finale ou effectivement finale). Enfin, les gestionnaires s'exécutent les uns après les autres sur le fil d'interface JavaFX : pas de risque que deux clics se mélangent, mais un traitement long dans un gestionnaire gèlerait la fenêtre — on garde donc les gestionnaires brefs.

L'essentiel
Programmation événementielle : setOnAction enregistre une lambda exécutée à chaque action de l'utilisateur. On y lit les champs (getText), on valide, on met à jour l'affichage (setText) ; l'état partagé vit dans les attributs de la classe.

Exemple — Compteur d'emprunts : trois gestionnaires en lambda

```
package com.iam.biblio.ui;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class CompteurEmprunts extends Application {
private int emprunts = 0;

@Override
public void start(Stage fenetre) {
Label affichage = new Label("Emprunts du jour : 0");
TextField champNom = new TextField();
champNom.setPromptText("Nom de l'emprunteur puis Entrée");
Button boutonEmprunter = new Button("Emprunter");
Button boutonZero = new Button("Réinitialiser");
boutonEmprunter.setOnAction(e -> { emprunts++;
affichage.setText("Emprunts du jour : " + emprunts); });
boutonZero.setOnAction(e -> { emprunts = 0;
affichage.setText("Emprunts du jour : 0"); });
champNom.setOnAction(e -> {
String nom = champNom.getText().trim();
if (!nom.isEmpty()) {
emprunts++;
affichage.setText(nom + " emprunte (total : " + emprunts + ")");
champNom.clear();
}
});
HBox boutons = new HBox(10, boutonEmprunter, boutonZero);
boutons.setAlignment(Pos.CENTER);
VBox racine = new VBox(12, affichage, champNom, boutons);
racine.setPadding(new Insets(15));
racine.setAlignment(Pos.CENTER);
fenetre.setTitle("Compteur d'emprunts");
fenetre.setScene(new Scene(racine, 360, 220));
fenetre.show();
}
public static void main(String[] args) { launch(args); }
}
```

À retenir

• setOnAction existe sur Button mais aussi sur TextField (déclenché par la touche Entrée).

• Une lambda qui modifie une variable extérieure exige un attribut de classe, pas une variable locale.

• Valider toujours la saisie dans le gestionnaire : trim() puis test isEmpty() avant de traiter.

Exercice

Enrichis le compteur d'emprunts : ajoute sous le champ de nom un label « Dernier emprunteur : — ». Quand un nom est validé par Entrée, ce label affiche le nom saisi. Ajoute aussi un bouton « Tout effacer » qui remet le compteur à zéro ET vide le champ de nom et le label du dernier emprunteur.

✔ Corrigé
Déclare le label dernier = new Label("Dernier emprunteur : —") et ajoute-le dans le VBox. Dans le gestionnaire du champ, après l'incrémentation, ajoute : dernier.setText("Dernier emprunteur : " + nom). Pour le bouton « Tout effacer », un seul gestionnaire fait les trois opérations : emprunts = 0 ; affichage.setText("Emprunts du jour : 0") ; champNom.clear() ; dernier.setText("Dernier emprunteur : —"). Remarque la méthode clear() de TextField, plus claire que setText(""). Un bon gestionnaire laisse toujours l'interface dans un état cohérent : compteur, champ et labels racontent la même histoire après le clic.

Leçon 4 — Tableaux, listes et formulaires graphiques

Pour afficher une collection d'objets, JavaFX fournit TableView<S>, où S est le type des lignes — chez nous, Livre. Chaque colonne est un TableColumn<Livre, Type> qu'il faut relier à une propriété de l'objet. Le mécanisme standard est PropertyValueFactory : new PropertyValueFactory<>("titre") cherche, pour chaque livre, la méthode getTitre() et affiche son résultat. Retiens la convention, source d'erreurs silencieuses : le nom passé est celui de la propriété (« disponible »), pas celui de la méthode — pour un booléen, JavaFX accepte getDisponible() ou isDisponible(), ce que notre classe Livre fournit. Si le nom est faux, la colonne reste vide sans aucune exception : en cas de colonne vide, vérifie d'abord l'orthographe de la propriété et l'existence du getter.

Les lignes du tableau vivent dans une ObservableList accessible par table.getItems() : toute modification de cette liste (add, remove) se répercute immédiatement à l'écran, sans code de rafraîchissement. Notre exemple ajoute au démarrage un livre écrit en dur, puis le formulaire permet d'en ajouter. Le formulaire est un GridPane : add(controle, colonne, ligne) — attention, la colonne vient en premier. Les écarts entre cellules se règlent par setHgap et setVgap. Le bouton « Ajouter » applique le schéma appris en leçon 3 : lire les champs, valider qu'ils ne sont pas vides, créer l'objet métier, l'ajouter à la liste observable, puis vider les champs pour la saisie suivante.

Pour agir sur une ligne précise, on lit la sélection : table.getSelectionModel().getSelectedItem() retourne l'objet sélectionné, ou null si aucune ligne n'est choisie — toujours tester ce null avant d'utiliser l'objet. C'est la porte d'entrée des fonctions « modifier » et « supprimer » (exercice ci-dessous et TD). Pour des listes simples de valeurs (des chaînes, des noms), ListView<T> fonctionne sur le même principe (getItems, getSelectionModel) avec une seule colonne implicite. TableView pour des objets à plusieurs propriétés, ListView pour des listes simples : choisir l'outil adapté fait partie du métier.

L'essentiel
TableView affiche des objets ; chaque TableColumn est reliée à une propriété par PropertyValueFactory (nom de propriété → getter). getItems() est la liste vivante du tableau ; getSelectionModel().getSelectedItem() donne la ligne choisie (ou null).

Exemple — TableView de livres relié au modèle, avec formulaire GridPane d'ajout

```
package com.iam.biblio.ui;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.stage.Stage;
import com.iam.biblio.modele.Livre;

public class TableLivres extends Application {
private <T> TableColumn<Livre, T> colonne(String titre, String propriete) {
TableColumn<Livre, T> col = new TableColumn<>(titre);
col.setCellValueFactory(new PropertyValueFactory<>(propriete));
return col;
}
@Override
public void start(Stage fenetre) {
TableView<Livre> table = new TableView<>();
table.getColumns().addAll(colonne("Titre", "titre"), colonne("Auteur", "auteur"),
colonne("Disponible", "disponible"));
table.getItems().add(new Livre(1, "L'Enfant noir", "Camara Laye"));
TextField champTitre = new TextField(), champAuteur = new TextField();
Button boutonAjouter = new Button("Ajouter à la liste");
boutonAjouter.setOnAction(e -> {
String t = champTitre.getText().trim();
String a = champAuteur.getText().trim();
if (!t.isEmpty() && !a.isEmpty()) {
table.getItems().add(new Livre(table.getItems().size() + 1, t, a));
champTitre.clear(); champAuteur.clear();
}
});
GridPane formulaire = new GridPane();
formulaire.setHgap(8); formulaire.setVgap(8);
formulaire.add(new Label("Titre :"), 0, 0); formulaire.add(champTitre, 1, 0);
formulaire.add(new Label("Auteur :"), 0, 1); formulaire.add(champAuteur, 1, 1);
formulaire.add(boutonAjouter, 1, 2);
VBox racine = new VBox(10, table, formulaire);
racine.setPadding(new Insets(12));
fenetre.setTitle("Catalogue de la bibliothèque");
fenetre.setScene(new Scene(racine, 520, 380)); fenetre.show();
}
public static void main(String[] args) { launch(args); }
}
```

À retenir

• PropertyValueFactory("titre") appelle getTitre() : une faute dans le nom = une colonne vide, sans erreur.

• GridPane.add(enfant, colonne, ligne) : la colonne est le premier indice.

• Après un ajout réussi, vider les champs (clear()) prépare la saisie suivante et évite les doublons.

Exercice

Ajoute à l'exemple un bouton « Supprimer » qui retire du tableau le livre actuellement sélectionné. Si aucune ligne n'est sélectionnée, le clic ne doit rien faire (et surtout ne pas planter). Place le bouton dans le formulaire, sous le bouton d'ajout.

✔ Corrigé
Le gestionnaire tient en quelques lignes : Livre choisi = table.getSelectionModel().getSelectedItem(); if (choisi != null) { table.getItems().remove(choisi); }. Le test de null est indispensable : sans sélection, getSelectedItem() retourne null et remove(null) ne supprimerait rien d'utile — pire, un appel de méthode sur choisi lèverait NullPointerException. Dans le GridPane, ajoute le bouton en ligne 3 (formulaire.add(boutonSupprimer, 1, 3)). Note que supprimer de la liste observable suffit : le TableView se met à jour tout seul, c'est exactement le bénéfice du modèle observable de JavaFX.

Leçon 5 — Connecter une application Java à une base de données avec JDBC

JDBC (Java Database Connectivity) est l'API standard de Java pour parler à une base SQL : elle fait partie du JDK (package java.sql). Entre ton code et le SGBD se glisse un pilote (driver) fourni par l'éditeur de la base — pour MySQL, c'est le fichier mysql-connector-j à ajouter au projet. La connexion s'ouvre avec DriverManager.getConnection(url, utilisateur, motDePasse), où l'URL a la forme jdbc:mysql://hôte:port/nom_base. Depuis JDBC 4.0, le pilote présent dans le classpath se charge tout seul : l'ancien Class.forName(...) n'est plus nécessaire. La classe qui regroupe toutes les requêtes d'une table s'appelle un DAO : c'est elle qui vit dans notre package dao (leçon 1).

Pour exécuter une requête contenant des données, on utilise PreparedStatement, jamais la concaténation de chaînes. La requête contient des points d'interrogation (les paramètres), que l'on remplit par setString, setInt, setBoolean… en commençant à l'indice 1. executeUpdate() sert aux INSERT/UPDATE/DELETE et retourne le nombre de lignes touchées ; executeQuery() sert aux SELECT et retourne un ResultSet que l'on parcourt avec next(), en lisant chaque colonne par son nom (getInt("id"), getString("titre")…). Au-delà de la commodité, le PreparedStatement est une protection : les valeurs transmises ne sont jamais interprétées comme du SQL, ce qui bloque l'injection SQL étudiée dans le cours full-stack (INF-L2-08).

Une connexion ouverte doit toujours être refermée, ainsi que les statements et les result sets — sinon la base finit par refuser de nouvelles connexions. La façon propre

de le garantir est le try-with-resources : les ressources déclarées dans les parenthèses du try sont fermées automatiquement, dans l'ordre inverse, même si une exception survient. Notre DAO l'applique partout : la connexion et le PreparedStatement dans le try externe, le ResultSet dans un try imbriqué pour les lectures. Les erreurs SQL lèvent SQLException, une exception vérifiée : chaque méthode du DAO la déclare (throws SQLException) et laisse l'interface décider comment la présenter à l'utilisateur (leçon 7).

L'essentiel
JDBC = java.sql + un pilote par SGBD + une URL jdbc:. PreparedStatement avec paramètres ? pour toutes les requêtes avec données (anti-injection), ResultSet pour lire les SELECT, try-with-resources pour fermer à coup sûr.

Exemple — LivreDAO : insertion et recherche par auteur avec PreparedStatement

```
package com.iam.biblio.dao;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import com.iam.biblio.modele.Livre;

public class LivreDAO {
private static final String URL = "jdbc:mysql://localhost:3306/biblio_iam";
private static final String UTILISATEUR = "biblio";
private static final String MOT_DE_PASSE = "biblio2026";

private Connection ouvrirConnexion() throws SQLException {
return DriverManager.getConnection(URL, UTILISATEUR, MOT_DE_PASSE);
}

public void inserer(Livre livre) throws SQLException {
String sql = "INSERT INTO livre (titre, auteur, disponible) VALUES (?, ?, ?)";
try (Connection cnx = ouvrirConnexion();
PreparedStatement ps = cnx.prepareStatement(sql)) {
ps.setString(1, livre.getTitre());
ps.setString(2, livre.getAuteur());
ps.setBoolean(3, livre.isDisponible());
ps.executeUpdate();
}
}

public List<Livre> rechercherParAuteur(String auteur) throws SQLException {
String sql = "SELECT id, titre, auteur, disponible FROM livre WHERE auteur = ?";
List<Livre> resultat = new ArrayList<>();
try (Connection cnx = ouvrirConnexion();
PreparedStatement ps = cnx.prepareStatement(sql)) {
ps.setString(1, auteur);
try (ResultSet rs = ps.executeQuery()) {
while (rs.next()) {
resultat.add(new Livre(rs.getInt("id"), rs.getString("titre"),
rs.getString("auteur")));
}
}
}
return resultat;
}
}
```

À retenir

- Les indices des paramètres d'un PreparedStatement commencent à 1, pas à 0.
- executeUpdate pour écrire (INSERT/UPDATE/DELETE), executeQuery pour lire (SELECT).
- Ne mets jamais de mot de passe réel dans un code partagé : ici, identifiants d'entraînement pour une base locale.

Exercice

Complète LivreDAO avec une méthode tousLesLivres() qui retourne la liste de tous les livres de la table, triés par titre croissant. Elle ne prend aucun paramètre. Écris la requête, le parcours du ResultSet et la fermeture des ressources dans le style de l'exemple.

✔ Corrigé
La méthode déclare : public List<Livre> tousLesLivres() throws SQLException. La requête est « SELECT id, titre, auteur, disponible FROM livre ORDER BY titre ». Comme il n'y a aucun paramètre, on peut ouvrir le ResultSet directement dans le même try-with-resources que la connexion et le PreparedStatement : try (Connection cnx = ouvrirConnexion(); PreparedStatement ps = cnx.prepareStatement(sql); ResultSet rs = ps.executeQuery()) { while (rs.next()) { resultat.add(new Livre(rs.getInt("id"), rs.getString("titre"), rs.getString("auteur"))); } } puis on retourne la liste. Astuce de lecture : quand un ResultSet est déclaré dans le try-with-resources, il est fermé avant le statement et la connexion, dans l'ordre inverse de leur ouverture — c'est exactement ce qu'exige JDBC. Cette méthode servira directement dans le projet de la leçon 7.

Leçon 6 — Empaquetage et distribution d'une application Java

Développer « chez soi » ne suffit pas : il faut livrer. La première forme de livraison est le jar exécutable : l'outil jar rassemble les .class (et les ressources) dans une archive dont le manifeste déclare la classe principale (Main-Class). L'utilisateur lance alors java -jar biblio-iam.jar. Mais il y a un piège : l'utilisateur doit avoir un Java compatible, et nos dépendances externes — le SDK JavaFX et le pilote MySQL — doivent aussi être présentes et trouvées au lancement. Un jar « qui marche sur ma machine » mais pas sur celle du bibliothécaire est un classique des premiers projets.

Le système de modules de Java (depuis Java 9) rend les dépendances explicites : le fichier module-info.java, placé à la racine des sources, déclare ce que notre module requiert (requires javafx.controls ; requires java.sql ;) et ce qu'il expose (exports com.iam.biblio.ui ;). En mode modulaire, la compilation et le lancement passent par le module path (--module-path) plutôt que le classpath. Avec des modules bien déclarés, l'outil jlink peut construire une image d'exécution sur mesure contenant uniquement les modules utilisés : plus légère que le JDK complet, et autonome.

L'étape finale est jpackage (inclus dans le JDK depuis Java 16) : il fabrique, à partir du jar et d'une image d'exécution, un livrable natif pour le système cible — image d'application ou installeur (sur Windows, la production d'un .exe/.msi demande en plus l'outil WiX ; sur Linux, .deb/.rpm ou une simple image). L'utilisateur final n'a alors rien à installer : il lance BiblioIAM comme n'importe quel logiciel. Avant de livrer, la discipline reste la même : tester sur une machine « propre » (sans ton environnement de développement), joindre le script SQL de création de la base, un fichier README avec les étapes d'installation, et un numéro de version.

L'essentiel
Livrer = jar exécutable (manifeste avec Main-Class) ou application native. module-info.java déclare requires/exports ; jlink taille une exécution sur mesure ; jpackage produit le livrable final que l'utilisateur lance sans Java préinstallé.

Exemple — module-info.java et les trois commandes de livraison de BiblioIAM

```
// Fichier module-info.java (à la racine des sources)
module com.iam.biblio {
requires javafx.controls;
requires java.sql;
exports com.iam.biblio.ui;
}

# Commandes (terminal, depuis la racine du projet)
# 1) Compiler en mode modulaire (JavaFX dans lib/)
javac --module-path lib/javafx-sdk-17/lib -d classes $(find src -name "*.java")
# 2) Créer le jar modulaire exécutable
jar --create --file biblio-iam.jar --main-class com.iam.biblio.ui.BiblioApp -C
classes .
# 3) Construire une image d'application autonome avec jpackage
jpackage --type app-image --name BiblioIAM --dest dist \
--module-path lib/javafx-sdk-17/lib:biblio-iam.jar \
--module com.iam.biblio/com.iam.biblio.ui.BiblioApp
```

À retenir

- java -jar exige un manifeste avec Main-Class et les dépendances accessibles : teste toujours sur une autre machine.
- En mode modulaire, --module-path remplace le classpath et module-info.java fait foi pour les dépendances.
- jpackage livre une application autonome ; l'installeur Windows exige WiX en plus du JDK.

Exercice

Une petite application console non modulaire « GestionNotes » est compilée dans un dossier bin/ ; sa classe principale est com.iam.notes.Main et elle n'utilise aucune bibliothèque externe. Donne la commande qui crée un jar exécutable nommé gestion-notes.jar, puis la commande qui le lance. Explique pourquoi aucun --module-path n'est nécessaire ici.

✔ Corrigé
Création : jar --create --file gestion-notes.jar --main-class com.iam.notes.Main -C bin . (le -C bin . signifie « prendre le contenu du dossier bin »). Lancement : java -jar gestion-notes.jar. Aucun module path n'est requis car l'application n'utilise que la bibliothèque standard du JDK et n'est pas modularisée : le jar fonctionne en mode classpath classique, et le manifeste généré par l'option --main-class suffit à désigner le point d'entrée. C'est la configuration la plus simple — dès qu'une bibliothèque modulaire comme JavaFX entre en jeu, on bascule vers la démarche de l'exemple.

Leçon 7 — Projet : application de bureau complète et documentée

Assemblons tout le cours dans BiblioApp, la fenêtre principale de BiblioIAM. Au démarrage, start() crée les deux colonnes du TableView, puis chargerLivres() demande la liste au DAO (la méthode tousLesLivres() écrite dans l'exercice de la leçon 5) et la verse dans le tableau avec FXCollections.observableArrayList(...). Le bouton « Ajouter » construit un Livre depuis les champs, appelle dao.inserer(...), puis recharge la table et vide les champs. Le flux de données est toujours le même, dans un seul sens : l'interface appelle le DAO, le DAO parle SQL, le modèle transporte les données. Aucune requête SQL n'apparaît dans le code de la fenêtre.

Une application livrable doit aussi gérer ses erreurs proprement. Ici, toute SQLException est attrapée et transformée en boîte de dialogue Alert de type ERROR avec un message compréhensible (« Enregistrement impossible : ... »), au lieu d'une trace d'exception illisible dans la console. La méthode afficherErreur centralise ce comportement. Il reste une limite assumée dans cette version : un titre vide serait envoyé à la base — les pistes d'amélioration sont connues (valider les champs comme en leçon 3 et afficher alors une Alert WARNING, ou désactiver le bouton tant que les champs sont vides) et font partie des prolongements du TD.

Enfin, documenter fait partie du livrable : un README qui explique les prérequis (MySQL démarré, base biblio_iam créée par le script fourni, table livre), comment lancer l'application et quelle version on livre ; des commentaires Javadoc sur les méthodes publiques du DAO ; et une dernière vérification sur une machine propre avec le paquet produit en leçon 6. Le TP qui suit te fait réaliser exactement ce parcours en trois séances. Tu sais désormais mener une application Java de la première classe métier jusqu'à l'installeur : c'est la compétence centrale visée par ce cours.

L'essentiel
L'application finale assemble les couches : la fenêtre ne contient aucun SQL, le DAO ne contient aucune interface, le modèle ne dépend de rien. Chaque erreur SQL devient un message utilisateur via Alert, et la livraison inclut README + script SQL + test sur machine propre.

Exemple — BiblioApp : la fenêtre complète reliée au DAO

```
package com.iam.biblio.ui;
import java.sql.SQLException;
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.stage.Stage;
import com.iam.biblio.dao.LivreDAO;
import com.iam.biblio.modele.Livre;

public class BiblioApp extends Application {
private final LivreDAO dao = new LivreDAO();
private final TableView<Livre> table = new TableView<>();
private static TableColumn<Livre, String> colonne(String t, String p) {
TableColumn<Livre, String> c = new TableColumn<>(t);
c.setCellValueFactory(new PropertyValueFactory<>(p)); return c;
}
private void chargerLivres() {
try { table.setItems(FXCollections.observableArrayList(dao.tousLesLivres())); }
catch (SQLException ex) { afficherErreur("Lecture impossible : " + ex.getMessage());
}
}
private void afficherErreur(String message) {
Alert a = new Alert(Alert.AlertType.ERROR);
a.setHeaderText(null); a.setContentText(message); a.showAndWait();
}
@Override
public void start(Stage fenetre) {
table.getColumns().addAll(colonne("Titre", "titre"), colonne("Auteur", "auteur"));
chargerLivres();
TextField champTitre = new TextField(), champAuteur = new TextField();
champTitre.setPromptText("Titre"); champAuteur.setPromptText("Auteur");
Button boutonAjouter = new Button("Ajouter");
boutonAjouter.setOnAction(e -> {
try {
dao.inserer(new Livre(0, champTitre.getText().trim(), champAuteur.getText().trim()));
chargerLivres(); champTitre.clear(); champAuteur.clear();
} catch (SQLException ex) { afficherErreur("Enregistrement impossible : " +
ex.getMessage()); }
});
HBox saisie = new HBox(8, champTitre, champAuteur, boutonAjouter); VBox racine = new
VBox(10, table, saisie);
fenetre.setTitle("BiblioIAM — catalogue"); fenetre.setScene(new Scene(racine, 560,
400)); fenetre.show();
}
public static void main(String[] args) { launch(args); }
}
```

À retenir

• Après chaque écriture en base (inserer), recharger la table : l'écran doit toujours refléter la base.

• Attraper SQLException au niveau de l'interface et la traduire en Alert : jamais de trace brute pour l'utilisateur.

• Le code de la fenêtre ne contient ni SQL ni logique métier : seulement affichage et appels au DAO.

Exercice

Ajoute à BiblioApp une recherche : un champ « Mot-clé » et un bouton « Rechercher » qui n'affichent que les livres dont le titre ou l'auteur contient le mot-clé (la méthode DAO correspondante est l'exercice supplémentaire 3). Un bouton « Tout afficher » rétablit la liste complète.

✔ Corrigé
Le gestionnaire du bouton « Rechercher » suit la même forme que celui d'« Ajouter » : try { table.setItems(FXCollections.observableArrayList(dao.rechercherParMotCle(champMotCle.getText().trim()))); } catch (SQLException ex) { afficherErreur("Recherche impossible : " + ex.getMessage()); }. Le bouton « Tout afficher » se contente d'appeler chargerLivres(). Deux points de vigilance : la recherche ne modifie pas la base, seulement le contenu affiché du tableau — un ajout effectué pendant qu'un filtre est actif doit donc recharger avec le même filtre ou réinitialiser l'affichage, sinon l'utilisateur croira son livre perdu ; et le mot-clé vide avec LIKE '%%' retourne tous les livres, ce qui est un comportement acceptable à condition de le savoir.

Travaux dirigés (TD)

TD 1 — Déboguer une fenêtre JavaFX fautive

Un camarade te montre ce code en affirmant que « la fenêtre ne s'ouvre même pas ». Trouve les QUATRE erreurs, explique pour chacune ce qui se passe (ou ne se passe pas), puis réécris la classe corrigée complète. public class MaFenetre { public void start(Stage fenetre) { TextField champ = new TextField(); Button bouton = new Button("Valider"); bouton.setOnAction(e -> { String texte = champ.getValue(); System.out.println("Saisie : " + texte); }); VBox racine = new VBox(10, champ, bouton); Scene scene = new Scene(fenetre); fenetre.setScene(scene); } }

✔ Corrigé du TD 1
Erreur 1 : la classe n'étend pas Application. La méthode start n'est alors jamais appelée par JavaFX — c'est juste une méthode ordinaire. Il faut « public class MaFenetre extends Application », marquer start avec @Override, et ajouter un main qui appelle launch(args). Erreur 2 : champ.getValue() n'existe pas sur TextField — le texte saisi se lit avec getText() (getValue appartient à d'autres contrôles comme les curseurs ou les listes déroulantes). Erreur 3 : new Scene(fenetre) passe le Stage au constructeur de Scene, alors qu'il attend le nœud racine (un Parent) : il faut new Scene(racine, 300, 150). Erreur 4 : fenetre.show() est absent : la fenêtre est construite et munie de sa scène, mais jamais affichée. Morale : trois de ces erreurs sont détectées par le compilateur (héritage/@Override, getValue, Scene(Stage)) — d'où la règle d'or : compiler souvent, par petits pas, plutôt que d'écrire 100 lignes avant le premier test.

TD 2 — Compléter le DAO : liste complète et suppression

Dans une classe LivreDAOComplet qui reprend les mêmes constantes de connexion que LivreDAO (leçon 5), écris : 1) tousLesLivres(), qui retourne tous les livres triés par titre ; 2) supprimer(int id), qui supprime le livre d'identifiant donné et retourne true si exactement une ligne a été supprimée, false sinon. Justifie le choix des méthodes d'exécution JDBC utilisées.

✔ Corrigé du TD 2
tousLesLivres() : requête « SELECT id, titre, auteur FROM livre ORDER BY titre », exécutée par executeQuery() (c'est une lecture) ; le ResultSet est déclaré dans le try-with-resources avec la connexion et le PreparedStatement, et chaque ligne devient un Livre ajouté à une ArrayList. supprimer(int id) : requête « DELETE FROM livre WHERE id = ? », paramètre rempli par ps.setInt(1, id), exécutée par executeUpdate() (c'est une écriture) qui retourne le nombre de lignes affectées : la méthode retourne (executeUpdate() == 1). Le booléen est précieux côté interface : false signifie « ce livre n'existait déjà plus », ce qui arrive si l'utilisateur clique deux fois ou si la base a été modifiée entre-temps — l'interface peut alors recharger la liste et prévenir l'utilisateur au lieu d'afficher un faux succès.

Travaux pratiques (TP)

TP 1 — BiblioIAM complète en trois séances

Séance 1 (base et modèle) : crée la base MySQL biblio_iam et la table livre (id INT PRIMARY KEY AUTO_INCREMENT, titre VARCHAR(150), auteur VARCHAR(100), disponible BOOLEAN DEFAULT TRUE) ; écris le package modele (Livre) et le DAO complet (inserer, rechercherParAuteur, tousLesLivres, supprimer) ; prouve le fonctionnement par un programme console qui insère 3 livres et affiche la liste. Séance 2 (interface) : construis BiblioApp (TableView, formulaire d'ajout, suppression de la ligne sélectionnée, recherche par mot-clé) avec les erreurs affichées par Alert. Séance 3 (livraison) : ajoute module-info.java, produis le jar puis une image d'application avec jpackage, écris le README (prérequis, script SQL, lancement) et teste le paquet sur la machine d'un camarade.

✔ Attendus du TP 1
Attendus vérifiables. Séance 1 : le programme console affiche les 3 livres insérés relus depuis la base (pas depuis une liste en mémoire) ; le DAO ne contient que des PreparedStatement. Séance 2 : ajout visible immédiatement dans le tableau ; suppression effective en base (contrôle par une requête SELECT dans le client MySQL) ; si MySQL est arrêté, l'application affiche une Alert claire et ne plante pas. Séance 3 : le dossier livré contient l'image jpackage, le script creation.sql et le README ; sur la machine du camarade, l'application démarre par double-clic et lit la base. Barème indicatif : modèle + DAO corrects et testés 7 pts ; interface complète et robuste 7 pts ; empaquetage fonctionnel 3 pts ; README et script SQL 3 pts.

Exercices corrigés supplémentaires

Exercice 1

Dans la fenêtre TableLivres (leçon 4), ajoute un bouton « Supprimer » dont le gestionnaire retire le livre sélectionné de la liste du tableau, en gérant le cas où rien n'est sélectionné. Donne le code du gestionnaire seul.

✔ Corrigé
boutonSupprimer.setOnAction(e -> { Livre choisi = table.getSelectionModel().getSelectedItem(); if (choisi != null) { table.getItems().remove(choisi); } }); — Deux idées à retenir : la sélection retourne l'objet métier lui-même (pas un numéro de ligne), donc remove() retire exactement le bon élément ; et le test de null transforme un clic « dans le vide » en non-événement au lieu d'un plantage. Ce code supprime de l'affichage seulement : dans l'application complète, on appellerait d'abord dao.supprimer(choisi.getId()) (TD 2) puis on retirerait la ligne si la base a confirmé.

Exercice 2

Écris le code de vérification d'un mini formulaire de connexion : un TextField champId pour l'identifiant, un PasswordField champMdp pour le mot de passe, et une méthode qui retourne « Champs obligatoires » si l'un des deux est vide, sinon « Bienvenue, » suivi de l'identifiant. Pourquoi utiliser PasswordField plutôt qu'un TextField ordinaire ?

✔ Corrigé
String id = champId.getText().trim(); String mdp = champMdp.getText(); if (id.isEmpty() || mdp.isEmpty()) { return "Champs obligatoires"; } return "Bienvenue, " + id; — PasswordField masque les caractères pendant la frappe (points à l'écran) pour qu'un mot de passe ne soit pas lu par-dessus l'épaule ; il hérite de TextField, donc getText() fonctionne de la même façon. Attention : masquer n'est pas chiffrer — la vérification réelle d'un mot de passe se fait côté base avec des empreintes (password_hash en PHP, fonction équivalente en Java), comme vu dans les cours de sécurité : ne stocke jamais un mot de passe en clair.

Exercice 3

Écris la méthode DAO rechercherParMotCle(String motCle) qui retourne les livres dont le titre OU l'auteur contient le mot-clé, en utilisant l'opérateur SQL LIKE et des paramètres préparés. Précise ce que vaut le motif recherché pour le mot-clé « noir ».

✔ Corrigé
La requête est « SELECT id, titre, auteur, disponible FROM livre WHERE titre LIKE ? OR auteur LIKE ? ». Le motif est construit en Java : String motif = "%" + motCle + "%"; puis ps.setString(1, motif); ps.setString(2, motif); — pour « noir », le motif vaut %noir%, c'est-à-dire « contient noir n'importe où ». Le parcours du ResultSet est identique à celui de rechercherParAuteur. Point de sécurité : le mot-clé passe par un paramètre préparé, donc même un mot-clé contenant des guillemets ou des % tapés par l'utilisateur ne peut pas détourner la requête (au pire, un % saisi élargit la recherche — comportement anodin et documenté).

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. À quoi sert principalement un package en Java ?

   A) À compresser les fichiers .class

   B) À regrouper les classes liées et à contrôler leur visibilité

   C) À accélérer l'exécution du programme

2. Toute application JavaFX doit hériter de la classe…

   A) Application

   B) Stage

   C) Scene

3. Dans « new Scene(racine, 520, 300) », les nombres 520 et 300 représentent…

   A) La position de la fenêtre à l'écran

   B) Le nombre de lignes et de colonnes

   C) La largeur et la hauteur de la scène, en pixels

4. La méthode setOnAction d'un bouton attend…

   A) Une chaîne de caractères

   B) Un gestionnaire d'événement (EventHandler), souvent une lambda

   C) Un entier

5. new PropertyValueFactory<>("titre") affiche dans la colonne…

   A) La valeur retournée par getTitre() de chaque objet

   B) Le champ privé titre, lu directement

   C) Le titre de la fenêtre

6. Le principal avantage de PreparedStatement sur la concaténation de chaînes SQL est…

   A) Il rend les requêtes plus courtes à écrire

   B) Il exécute les requêtes sans connexion

   C) Il empêche l'injection SQL en traitant les valeurs comme des données, jamais comme du code

7. Le try-with-resources garantit que…

   A) Les requêtes sont plus rapides

   B) Les ressources (Connection, Statement, ResultSet) sont fermées automatiquement, même en cas d'exception

   C) La base de données est créée si elle n'existe pas

8. La méthode next() d'un ResultSet…

   A) Avance le curseur d'une ligne et retourne true s'il y a une ligne à lire

   B) Passe à la table suivante

   C) Ferme le résultat

9. L'outil jpackage sert à…

A) Compiler le code Java

B) Écrire des requêtes SQL

C) Produire un livrable natif (image d'application ou installeur) embarquant une exécution Java

10. Le fichier module-info.java déclare…

A) Les utilisateurs de l'application

B) Les modules dont l'application dépend (requires) et les packages qu'elle expose (exports)

C) Les requêtes SQL du projet

Corrigés du QCM

1. Réponse B — Un package est un espace de noms et une frontière de visibilité : il organise le code et protège ce qui n'est pas public.

2. Réponse A — La classe principale étend javafx.application.Application et redéfinit start(Stage) ; main appelle launch(args).

3. Réponse C — Le constructeur Scene(Parent, largeur, hauteur) fixe la taille initiale du contenu de la fenêtre.

4. Réponse B — setOnAction(EventHandler<ActionEvent>) enregistre le code à exécuter lors du clic ; la lambda est la forme la plus courante.

5. Réponse A — PropertyValueFactory cherche la propriété « titre » et appelle son accesseur public getTitre() (ou titreProperty()).

6. Réponse C — Les paramètres ? sont transmis séparément de la requête : une valeur malveillante ne peut pas modifier le SQL.

7. Réponse B — Toute ressource AutoCloseable déclarée dans le try est fermée à la sortie du bloc, dans l'ordre inverse de l'ouverture.

8. Réponse A — On parcourt les résultats avec while (rs.next()) { ... } : le curseur démarre avant la première ligne.

9. Réponse C — jpackage transforme le jar et une image d'exécution en application que l'utilisateur lance sans installer Java.

10. Réponse B — C'est le descripteur du module : il rend les dépendances explicites et vérifiables par le compilateur et jlink.

Résumé — les notions importantes

• Organisation en packages : modele (données métier), dao (accès aux données), ui (fenêtres) — dépendances dans un seul sens, attributs private, classes partagées en public.

• JavaFX : la classe principale étend Application ; start(Stage) construit une Scene faite d'un arbre de nœuds et l'affiche par show().

• Les conteneurs disposent les contrôles : BorderPane (5 zones), VBox/HBox (empilements), GridPane (formulaires en grille) — jamais de coordonnées absolues.

• Événements : setOnAction + lambda pour réagir aux clics et à la touche Entrée ; lire avec getText, écrire avec setText, valider avant de traiter, état conservé dans les attributs.

• TableView : chaque colonne est reliée à une propriété par PropertyValueFactory ; getItems() est la liste vivante ; la sélection se lit par getSelectionModel().getSelectedItem() (null possible).

• JDBC : DriverManager + URL jdbc:, PreparedStatement paramétré pour toutes les données (anti-injection), executeQuery/executeUpdate selon le sens, ResultSet parcouru par next(), fermeture systématique par try-with-resources.

• Distribution : jar exécutable (Main-Class), modules (module-info, jlink) et jpackage pour livrer une application autonome, accompagnée du script SQL et d'un README, testée sur une machine propre.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Rappels de POO et organisation du code en packages
La déclaration package est la première ligne du fichier, et les dossiers doivent refléter son chemin. ; Une classe utilisée depuis un autre package doit être public et importée ; les attributs restent private. ; Architecture en couches : ui → dao → modele, jamais l'inverse.

Fiche 2 — Interfaces graphiques avec JavaFX : conteneurs et contrôles
start(Stage) est le point d'entrée : on y construit la scène puis on appelle show(). ; Un contrôle ne s'affiche que s'il est ajouté dans un conteneur lui-même placé dans la scène. ; setPromptText affiche un texte d'aide dans un champ vide ; il ne remplace pas une étiquette.

Fiche 3 — Gestion des événements dans une interface graphique
setOnAction existe sur Button mais aussi sur TextField (déclenché par la touche Entrée). ; Une lambda qui modifie une variable extérieure exige un attribut de classe, pas une variable locale. ; Valider toujours la saisie dans le gestionnaire : trim() puis test isEmpty() avant de traiter.

Fiche 4 — Tableaux, listes et formulaires graphiques
PropertyValueFactory("titre") appelle getTitre() : une faute dans le nom = une colonne vide, sans erreur. ; GridPane.add(enfant, colonne, ligne) : la colonne est le premier indice. ; Après un ajout réussi, vider les champs (clear()) prépare la saisie suivante et évite les doublons.

Fiche 5 — Connecter une application Java à une base de données avec JDBC
Les indices des paramètres d'un PreparedStatement commencent à 1, pas à 0. ; executeUpdate pour écrire (INSERT/UPDATE/DELETE), executeQuery pour lire (SELECT). ; Ne mets jamais de mot de passe réel dans un code partagé : ici, identifiants d'entraînement pour une base locale.

Fiche 6 — Empaquetage et distribution d'une application Java
java -jar exige un manifeste avec Main-Class et les dépendances accessibles : teste toujours sur une autre machine. ; En mode modulaire, --module-path remplace le classpath et module-info.java fait foi pour les dépendances. ; jpackage livre une application autonome ; l'installeur Windows exige WiX en plus du JDK.

Fiche 7 — Projet : application de bureau complète et documentée
Après chaque écriture en base (inserer), recharger la table : l'écran doit toujours refléter la base. ; Attraper SQLException au niveau de l'interface et la traduire en Alert : jamais de trace brute pour l'utilisateur. ; Le code de la fenêtre ne contient ni SQL ni logique métier : seulement affichage et appels au DAO.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 2 — Introduction | JavaFX - EP1

Chaîne : EvoluNoob · Langue : Français · Niveau : Débutant

Premier épisode d'une série complète sur JavaFX : présentation de la bibliothèque, structure habituelle d'un projet et découverte des composants d'interface. Idéal avant la leçon 2 pour installer l'environnement et voir à quoi ressemblera le résultat.

https://www.youtube.com/watch?v=s-Dt5FluAFw

Leçon 3 — JavaFX #8 - évènements

Chaîne : EvoluNoob · Langue : Français · Niveau : Débutant à intermédiaire

Le fonctionnement complet des événements JavaFX : filtres et gestionnaires, classes anonymes, méthodes pratiques et exemples au clavier et à la souris. Le prolongement direct de la leçon 3, pour dépasser le simple setOnAction.

https://www.youtube.com/watch?v=GDGB1z-Co5U

Leçon 4 — Fragments JavaFX

Chaîne : Professeur Cot · Langue : Français · Niveau : Intermédiaire

Comment découper une interface en fragments réutilisables pour garder un code d'affichage lisible quand les formulaires grandissent. À regarder après la leçon 4 pour structurer les fenêtres plus riches que de simples listes.

https://www.youtube.com/watch?v=g18AZZIglRI

Leçon 5 — JDBC Java Connexion Base de données MySQL

Chaîne : Masseka Academy · Langue : Français · Niveau : Débutant

La connexion d'une application Java à une base MySQL pas à pas : pilote JDBC, ouverture de la connexion, exécution de requêtes et lecture des résultats. À suivre en codant en parallèle de la leçon 5.

https://www.youtube.com/watch?v=p4_S0z0doOs

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Documentation officielle JavaFX (openjfx.io) : guide de démarrage, tutoriels sur les conteneurs et les contrôles, et la Javadoc des packages javafx.scene.control et javafx.scene.layout.

• Oracle — tutoriel « JDBC Basics » et documentation de l'API java.sql (docs.oracle.com/javase/tutorial/jdbc) : connexion, PreparedStatement, ResultSet et gestion des transactions.

• Documentation des outils du JDK 17 (docs.oracle.com) : jar (création d'archives exécutables), jlink (images d'exécution) et jpackage (empaquetage d'applications).

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !